Cooperative Vehicle Speed Fault Diagnosis and Correction

Mohammad Pirani, Ehsan Hashemi, Amir Khajepour, Baris Fidan, Bakhtiar Litkouhi, Shih-Ken Chen, and Shreyas Sundaram

***Abstract*— Reliable estimation of vehicle speed is an active topic of research in the automotive industry and academia due to its technical challenges as well as applications to vehicle traction and stability control. In this direction, the emergence of new generations of communication technologies has brought new perspectives to traditional studies on vehicle speed estimation and control. To this end, this paper introduces a cooperative vehicle speed fault diagnosis and correction algorithm. The distributed part of the algorithm is based on a distributed function calculation algorithm for vehicle networks. The introduced algorithm enables each vehicle to gather some information from other vehicles in the network in a distributed manner and is robust to communication failures. A procedure to use such information for a single vehicle to diagnose and correct a possible fault in its own speed estimation/measurement is discussed. The functionality and performance of the proposed algorithms are verified via illustrative examples and simulation results.**

***Index Terms*— Distributed calculation, vehicle platoons, speed fault diagnosis, fault tolerant systems.**

I. Introduction

The emergence of the *Internet of Vehicles*, as a tangible representation of the Internet of Things, has significantly changed the shape of urban transportation [1]–[3]. The rate of growth of this field of research has become so high that it is expected that the traffic of Internet of vehicles will double in the next few years [1]. The advent of ever-growing vehicle-to-vehicle (V2V) communications enable drivers to transmit and receive necessary information which can be potentially used for traffic management, fuel efficiency, as well as increasing the reliability of individual vehicle state estimation and control [4]–[6]. In particular, vehicle active safety systems can take advantage of the extra information obtained from the network of connected vehicles to design more reliable vehicle velocity estimation algorithms, which has been the focus of significant research during the past decade [7], [8].

Vehicle velocities, in both longitudinal and lateral directions, play fundamental roles in traction and stability control systems. They can be directly measured using Global Positioning System (GPS); however, the insufficient precision of conventional GPSs and their occasional signal drop issues are the drawbacks of this method. Moreover, implementing high-accuracy GPSs imposes a high production cost, which makes it infeasible to use. Thus, an alternative approach is to estimate the velocity, based on available sensor data instead of direct measurement [9], [10].

Manuscript received November 1, 2017; revised January 14, 2018; accepted March 24, 2018. Date of publication May 17, 2018; date of current version January 31, 2019. The Associate Editor for this paper was J. E. Naranjo. *(Corresponding author: Mohammad Pirani.)*

M. Pirani, E. Hashemi, A. Khajepour, and B. Fidan are with the Department of Mechanical and Mechatronics Engineering, University of Waterloo, Waterloo, ON N2L 3G1, Canada (e-mail: mpirani@uwaterloo.ca; ehashemi@uwaterloo.ca; a.khajepour@uwaterloo.ca; fidan@uwaterloo.ca).

B. Litkouhi and S.-K. Chen are with the Research and Development Department, General Motors Company, Warren, MI 48093 USA.

S. Sundaram is with the School of Electrical and Computer Engineering, Purdue University, West Lafayette, IN 47907 USA (e-mail: sundara2@purdue.edu).

Digital Object Identifier 10.1109/TITS.2018.2820044

Unfortunately, there are few efficient distributed calculation algorithms[1] for vehicle ad-hoc networks, compatible with the current available inter-vehicle communication standards. To this end, this paper applies a distributed calculation algorithm to obtain vehicles' states (position and speed) from other vehicles in the network and proposes a procedure to diagnose and correct faults (if any) in the estimation of the speed of any single vehicle.

Distributed calculation techniques have been widely investigated during the past decade [12]–[16]. The essence of distributed function calculation is that each member of the network calculates some quantity associated with all other members in the network (even from the members which are not directly connected to) via only local interactions (communications with neighbors) [11], [17]–[19]. More concisely, some global information is obtained via using local interactions [20], [21]. One of the main contributions of distributed calculation algorithms is to translate the system-theoretic notions of observability into network-theoretic concepts. In this direction, this paper uses one of the developed distributed calculation algorithms which is compatible with vehicle ad-hoc networks to estimate vehicle speeds in a distributed manner. It also proposes an algorithm for utilizing such globally gathered network information to diagnose possible failures in speed measurement (or estimation) of each single vehicle. We should note that an accurate estimation of vehicle speed has been an active topic of research due to its technical difficulties [8], [10] and the objective of this paper is to increase the reliability of the vehicle speed estimation with the availability of information obtained from other vehicles in the network.

The contributions of this paper are:

1) We apply a distributed calculation algorithm to a network of connected vehicles. Based on the sufficient condition proposed for the algorithm, we introduce a class of vehicle networks, called k-nearest neighbor platoons, which guarantees a specific level of connectivity. The proposed distributed calculation methodology is robust to communication failures and enables reliable vehicle state estimation, which is essential in automotive applications.
2) We discuss ways to use information (position and speed) gathered from the distributed calculation module to diagnose and correct faults in the measured (or estimated) speed of any single vehicle in the network.

II. Notation and Definitions

This section introduces the mathematical notation used in this paper, and provides a brief overview of vehicle-to-vehicle (V2V) communications.

A. Notation

In this paper, an undirected network is denoted by $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, where $\mathcal{V} = \{v_1, v_2, \ldots, v_n\}$ is a set of nodes (or vertices) and

[1]These algorithms are sometimes referred to as *distributed estimation algorithms*; however, in this paper we use distributed calculation as used in [11] in order to avoid confusion with single vehicle state estimation.

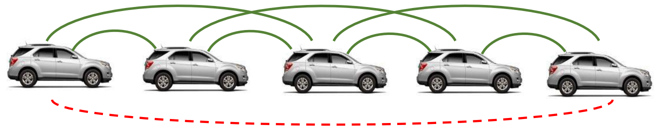

Fig. 1. A 2-nearest neighbor platoon of 5 vehicles, $\mathcal{P}(5, 2)$.

$\mathcal{E} \subset \mathcal{V} \times \mathcal{V}$ is the set of edges. Neighbors of node $v_i \in \mathcal{V}$ are given by the set $\mathcal{N}_i = \{v_j \in \mathcal{V} \mid (v_i, v_j) \in \mathcal{E}\}$. The degree of each node v_i is denoted by $d_i = |\mathcal{N}_i|$. A k-nearest neighbor platoon, $\mathcal{P}(n, k)$, is a network comprised of n nodes (here vehicles) in a string, where the nodes are labeled as $v_1, v_2, \ldots, v_n$ from one end of the string to the other and each node v_i can communicate with its k nearest neighbors behind it and k nearest neighbors ahead of it, i.e., with $v_{i-k}, v_{i-k+1}, \ldots, v_{i-1}, v_{i+1}, v_{i+2}, \ldots, v_{i+k}$ for some $k \in \mathbb{N}$. An example of $\mathcal{P}(n, k)$ is shown in Fig. 1. This definition is compatible with vehicle networks, due to the limited sensing and communication range for each vehicle and the distance between the consecutive vehicles [22]. This specific network topology, other than its applicability in analyzing vehicle-to-vehicle communication scenarios, has a particular network property, called k-connectivity, which will be discussed and used in Section VI.

B. V2V Communications

The Dedicated Short Range Communications for Wireless Access in Vehicular Environments (DSRC-WAVE), or simply DSRC,[2] are one-way or two-way short-range to medium-range wireless communication channels that are specifically designed for automotive applications. DSRC works in the 5.9 GHz band, uses a 75 MHz bandwidth channel, and provides low latency communication for safety applications [23], [24]. However, existing data sources and communications media (e.g. mobile phone and radio frequency) could be used for non-safety applications as well. The range of coverage of such communication is up to 1000 meters and the rate of data transmission is up to 27 Mbps. One of the standards which supports some DSRC applications is SAE J2735 [25]. Standard SAE J2735 specifies a message set and its data frames and data elements specifically for use by applications intended to utilize the 5.9 GHz. This message set has the potential to be used for applications that may be deployed in conjunction with other wireless communications technologies. There are various pieces of information that are sent via SAE J2735, including (but not limited to): temporal ID of the vehicle, time of transmission, latitude, longitude and elevation of the vehicle, longitudinal speed and acceleration of the vehicle, and break system status. In this paper, we will use the following: (i) vehicle v_i's position (called p_i) and (ii) its longitudinal velocity (speed), u_i.

III. Problem Statement

Consider a network of connected vehicles $v_1, v_2, .., v_n$ which is represented via a graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$. These vehicles can communicate with each other and share or disseminate information throughout the network. As mentioned in Section II, we only focus on the case where vehicles send their positions and speeds (longitudinal velocities) to each other. Each vehicle v_i is able to estimate its longitudinal velocity, u_i, via a specific method.[3] The goal for vehicle v_i is to assess the correctness of its speed u_i, utilizing an independent source of information. Such an independent source of information can be the values (positions and velocities) of other vehicles, together with an algorithm to relate these obtained values to each other. Therefore, a distributed calculation algorithm is introduced which enables vehicle v_i to gather the positions and speeds of other vehicles in the network and then assess the reliability of its own speed based on the gathered data.

[2]In Europe ETSI-ITS (European Telecommunications Standards Institute-Intelligent Transportation Systems) is used for inter-vehicular communications. DSRC and ETSI are both developed from standard IEEE 802.11p for the wireless local area networks.

[3]The velocity estimation method of each single vehicle is out of the scope of this paper and the reader is encouraged to see [7]–[10].

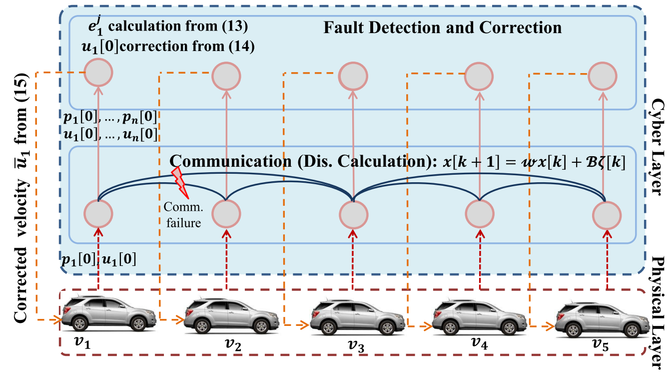

Fig. 2. Cyber-Physical representation of the proposed distributed fault diagnosis algorithm.

Remark 1 (Importance of Reliable Speed Estimation): Vehicle velocities, in both longitudinal and lateral directions, play fundamental roles in traction and stability control systems. They can be directly measured using Global Positioning Systems (GPS); however, the insufficient precision of conventional GPSs and their occasional signal drop issues are the drawbacks of this method. Moreover, implementing high-accuracy GPSs imposes a high production cost. Thus, an alternative approach is to estimate the velocity, based on available sensor data instead of direct measurement [9], [10]. In this direction, a feasible method is to utilize information from the cloud or from the network of connected vehicles (which already exists with no cost) to enhance the reliability and fault detection of the estimated velocity. □

The overall procedure of the cooperative vehicle speed fault diagnosis is:

(1) **Distributed Calculation:** Each vehicle v_i applies a distributed algorithm to gather the information of all of the vehicles in the network by using only local information exchange (with the vehicles in its communication range, called neighbors). Two different versions of this algorithm are proposed in this paper. In the first algorithm, it is assumed that there is no failure in vehicle communications, Section IV. A, and in the second algorithm, we assume that some of the vehicles fail to disseminate their information properly, Section IV. C.

(2) **Speed Fault Diagnosis:** After obtaining the information from other vehicles, vehicle v_i executes an algorithm, based on (relative) positions and velocities, to determine whether its own velocity estimate is faulty or correct.

(3) **Speed Correction:** If vehicle v_i determines that its self-speed-measurement is faulty (based on the previous step), it uses the information of other vehicles in the network to correct its own velocity estimate. In particular, v_i calculates its speed from the perspective of the other vehicles in the network.

Remark 2 (Cyber-Physical Interpretation): Fig. 2 provides a cyber-physical interpretation of the above steps. In particular, all the above three steps are developed in the cyber layer, which receives the physical states of vehicles from the physical layer as initial conditions for its algorithm (red dashed lines) and returns the corrected velocity back to the physical layer at the end

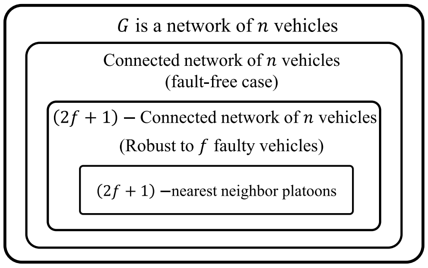

Fig. 3. Venn diagram of the vehicle network connectivity required for distributed calculation, based on Proposition 1 and Theorems 1, 3.

(orange dashed lines). Fig. 3 shows a Venn diagram of connectivity measures for vehicle networks in the cyber-layer, which will be discussed in detail later in the paper. □

We should note that the main advantage of the proposed algorithm compared to the other V2V algorithms, e.g., Geo-Broadcast [26], is its resilience to communication failures (or signal drop as a special case) and even attacks. Thus, our proposed algorithm is a modified version of the existing algorithms in the sense of security and communication fault tolerance.

In the following section, we begin by introducing distributed calculation algorithm (step 1). The fault diagnosis and speed correction schemes (steps 2 and 3) will be discussed in Section V.

IV. Robust Distributed Calculation

In this section we introduce an algorithm for a vehicle to gather the velocities of all vehicles in a network in a distributed manner, for two cases: (i) fault-free communications, and (ii) fault-prone-communications. The theoretical foundations of these algorithms are derived in [11] and [17]. The state of vehicle v_j, which can be its position p_j or speed u_j in this paper, is denoted simply by scalar $x_j[0]$. The objective is to enable vehicle v_i in the network (which is not in the communication range of vehicle v_j) to calculate this value. To yield this, vehicle v_i performs a linear iterative policy using the following time invariant updating rule

$$x_i[(k+1)T] = w_{ii}x_i[kT] + \sum_{j\in\mathcal{N}_i} w_{ij}x_j[kT], \tag{1}$$

where $w_{ii}, w_{ij} > 0$ are some predefined weights. From now we drop the sampling time T from the equations, unless it is necessary to be indicated.

The dynamics (1) can be written in the vector form as

$$\mathbf{x}[k+1] = \mathcal{W}\quad \mathbf{x}[k], \tag{2}$$

where $\mathcal{W}_{n\times n}$ is a matrix which captures the communication between vehicles in the network. In addition to dynamics (1), at each time step, vehicle v_i has access to its own value and the values of its neighbors. Hence, the output measurements for v_i is defined as

$$y_i[k] = \mathcal{C}_i\mathbf{x}[k], \tag{3}$$

where $\mathcal{C}_i$ is a $(d_i+1)\times n$ matrix with a single 1 in each row that denotes the positions of the state-vector $\mathbf{x}[k]$ available to vehicle v_i (i.e., these positions correspond to vehicles that are neighbors of v_i, along with vehicle v_i itself). The aim of such information dissemination is for each vehicle to obtain the initial condition of all other vehicles after running the linear dynamics (2) after some time steps.

Remark 3: It should be again noted that state $\mathbf{x}[k]$ in (2) evolves in the cyber layer (as shown in Fig. 2) and does not represent the evolution of vehicle's position or speed based on the communication; the dynamics (2) is only used for implementing a distributed calculation algorithm. Here, it is only $\mathbf{x}[0]$ that reflects the physical states of the vehicles. Moreover, in this study, we consider the fact that the communication between vehicles, dynamics (2), is much faster than the motion of the vehicle. In particular, although calculated quantities (positions and speeds) obtained from the distributed algorithm are available after few time steps in inter-vehicle communications, such a time delay is negligible due to the relatively slow physical motion of the vehicles compared to the inter-vehicular data transmission rate. □

Now suppose that there exist some vehicles that do not obey (2) to update their value. More formally, at time step k, vehicle v_i deviates from the predefined policy (1) and adds an arbitrary value, $\zeta_i[k]$, to its updating policy which yields

$$x_i[k+1] = w_{ii}x_i[k] + \sum_{j\in\mathcal{N}_i} w_{ij}x_j[k] + \zeta_i[k]. \tag{4}$$

Here, $\zeta_i[k]$ can represent a faulty or malicious update by vehicle v_i at time step k. This is written in vector form as

$$\mathbf{x}[k+1] = \mathcal{W}\mathbf{x}[k] + \underbrace{[\mathbf{e}_1\quad \mathbf{e}_2\quad \dots\quad \mathbf{e}_f]}_{\mathcal{B}}\boldsymbol{\zeta}[k], \tag{5}$$

where $\boldsymbol{\zeta}[k] = [\zeta_1[k], \zeta_2[k], \dots, \zeta_f[k]]^T$ is the vector of f faulty inputs and $\mathbf{e}_j$ denotes an $n\times 1$ unit vector with a single nonzero entry with value 1 at its j-th position. From this view, dynamics (2) is a special case of (5) when there is no faulty vehicle. The continuous-time version of the above network dynamics is studied extensively [27], [28]. The set of faulty vehicles in (5) is unknown and consequently matrix $\mathcal{B}$ is unknown. The objective is for vehicle v_i to calculate the true initial values of all other vehicles, despite the actions of the faulty vehicles. Based on (5), the set of all values seen by vehicle v_i during the first $L+1$ time-steps of the linear iteration is given by

$$\underbrace{\begin{bmatrix} y_i[0] \\ y_i[1] \\ \vdots \\ y_i[L] \end{bmatrix}}_{y_i[0:L-1]} = \underbrace{\begin{bmatrix} \mathcal{C}_i \\ \mathcal{C}_i\mathcal{W} \\ \vdots \\ \mathcal{C}_i\mathcal{W}^L \end{bmatrix}}_{\mathcal{O}_{i,L}} \mathbf{x}[0] + \underbrace{\begin{bmatrix} 0 & 0 & \cdots & 0 \\ \mathcal{C}_i\mathcal{B} & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \mathcal{C}_i\mathcal{W}^{L-1}\mathcal{B} & \mathcal{C}_i\mathcal{W}^{L-2}\mathcal{B} & \cdots & \mathcal{C}_i\mathcal{B} \end{bmatrix}}_{\mathcal{M}^{\mathcal{I}}_{i,L}} \underbrace{\begin{bmatrix} \boldsymbol{\zeta}[0] \\ \boldsymbol{\zeta}[1] \\ \vdots \\ \boldsymbol{\zeta}[L] \end{bmatrix}}_{\boldsymbol{\zeta}[0:L-1]} \tag{6}$$

where $\mathcal{O}_{i,L}$ and $\mathcal{M}^{\mathcal{I}}_i$ are called observability and invertability matrices, respectively and $\mathcal{I}$ is the set of faulty vehicles which form matrix $\mathcal{B}$ in (5). In order to estimate the initial states, the network dynamical system (5) together with the output measurement (3) should satisfy certain observability conditions. Thus, we analyze the distributed state estimation for the fault-free case (2) and then generalize this to the case where some vehicles update their states with some faults (5).

Remark 4: The distributed calculation algorithm in the presence of vehicle communication fault analyzed in this paper contains the scenario where a vehicle stops receiving a signal from its neighbors. This scenario is called *signal packet drop* in the communication literature [29]–[31]. More formally, in (4) if we set $\zeta_i[k] = -\sum_{j\in\mathcal{N}_i} w_{ij}x_j[k]$, it becomes equivalent to the case where v_i does not receive data from its neighbors. Since the analysis in this paper does not depend on the value of $\zeta_i[k]$, the packet dropping scenario can be straightforwardly included in the robust distributed calculation analysis. □

In the following subsection, we discuss the observability of dynamics (2) and output equation (3) when there are no faults in inter-vehicle communications.

A. Fault-Free Case, Dynamics (2)

For the case where there is no fault in inter-vehicular communications, the second term in (6) (containing $\boldsymbol{\zeta}[0 : L-1]$) does not exist. In this case, for each vehicle υ_i to be able to observe the initial conditions of other vehicles via its measurements, the system should be simply observable, i.e., its observability matrix $\mathcal{O}_{i,L}$ should be full row rank [32]. More formally, if vehicle υ_i wants to observe the initial condition of vehicle υ_j in the network after L time steps, the row space of $\mathcal{O}_{i,L}$ should contain $\mathbf{e}_j$. The following theorem introduces the freedom in designing the weight matrix $\mathcal{W}$ such that the observability matrix is guaranteed to be full row rank.

Theorem 1 [17]: Let $\mathcal{G}(\mathcal{V}, \mathcal{E})$ be a fixed and connected graph and define

$$\mathcal{S}_i = \{\upsilon_j | \text{There exists a path from } \upsilon_j \text{ to } \upsilon_i \text{ in } \mathcal{G}\} \cup \upsilon_i.$$

Moreover, let ϵ_i be the distance of the farthest vehicle in the network to υ_i. Then for almost[4] any choice of weights in the matrix $\mathcal{W}$, vehicle υ_i can obtain the initial value of vehicle $\upsilon_j \in \mathcal{S}_i$ after running the linear iteration (2) and output equation (3) for at least ϵ_i and at most $|\mathcal{S}_i| - d_i$ time steps. □

In order to implement the distributed calculation algorithm, each vehicle must have access to the observability matrix to calculate the initial conditions. More precisely, in order to find the initial condition of vehicle υ_j, $\mathbf{e}_j$ should be in the row space of the observability matrix. Hence, in order to find the initial condition of all vehicles, $I_{n\times n}$ should be in the row space of the observability matrix $\mathcal{O}_{i,L}$. If this condition is satisfied, vehicle υ_i can find a matrix Γ_i such that

$$\Gamma_i \mathcal{O}_{i,L} = I_{n\times n}, \tag{7}$$

and based on (6) the vector of initial conditions will be obtained.

Remark 5: The advantage of the proposed distributed calculation algorithm, compared to flooding algorithm, in which every incoming packet is sent through every outgoing link except the one it arrived on, is that the flooding would require the packet sizes to increase (since each vehicle would transmit all new information it received in the previous round); moreover, it requires each vehicle to transmit multiple times (sending different information to different neighbors). However, the proposed algorithm does not require increasing the packet size and multiple transmissions. □

Based on (7), each vehicle in the network should calculate the observability matrix distributedly to be able to calculate matrix Γ_i. This is discussed in the following subsection.

B. Distributed Calculation of the Observability Matrix

Here we introduce a distributed algorithm to calculate the observability matrix, based on [17]. Once the set of weights for matrix $\mathcal{W}$ in (2) is chosen, each vehicle performs n runs of the linear iteration, each for $n - d_{min}$ time-steps. For the j–th run, vehicle υ_j sets its initial condition to be 1, i.e. $\bar{\mathbf{x}}_j[0] = \mathbf{e}_j$, and all other nodes set their initial conditions to be zero. After performing all runs, each vehicle υ_i has access to matrix

$$\Psi_i = \begin{bmatrix} y_{i,1}[0] & y_{i,2}[0] & \cdots & y_{i,n}[0] \\ y_{i,1}[1] & y_{i,2}[1] & \cdots & y_{i,n}[1] \\ \vdots & \vdots & \ddots & \vdots \\ y_{i,1}[k_i] & y_{i,2}[k_i] & \cdots & y_{i,n}[k_i] \end{bmatrix}, \tag{8}$$

where $k_i = n - d_i - 1$ and $y_{i,j}[k] = \mathcal{C}_i \bar{\mathbf{x}}_j[k]$. Using (6) Ψ_i can be written as

$$\Psi_i = \mathcal{O}_{i,n-d_i-1}[\bar{\mathbf{x}}_1[0], \bar{\mathbf{x}}_2[0], \ldots, \bar{\mathbf{x}}_n[0]], \tag{9}$$

and since $\bar{\mathbf{x}}_i[0] = \mathbf{e}_i$ we conclude that $\Psi_i = \mathcal{O}_{i,n-d_i-1}$. Hence, each vehicle obtains its observability matrix in a distributed way. From this, each vehicle is able to calculate matrix Γ_i from (7).

C. Fault-Prone Case, Dynamics (5)

In this subsection, we extend the discussion in the previous subsection to the case where there are some vehicles which fail to communicate properly. Such failures can result in preventing other vehicles from running the distributed calculation algorithm properly. In this case, the values measured by vehicle υ_i is given by (6) where $\boldsymbol{\zeta}[0 : L-1]$ is no longer zero for faulty vehicles. Hence, one should design an algorithm to find the initial conditions despite of the actions of these faulty vehicles. This clearly demands a stronger condition on the system observability, as discussed in the following theorem.

Theorem 2 [11]: Suppose that there exists an integer L and a weight matrix $\mathcal{W}$ such that, for all possible sets of f faulty vehicles $\mathcal{I}$, the matrices $\mathcal{O}_{i,L}$ and $\mathcal{M}^{\mathcal{I}}_{i,L}$ for vehicle υ_i satisfy

$$\operatorname{rank}\left([\mathcal{O}_{i,L} \quad \mathcal{M}^{\mathcal{I}}_{i,L}]\right) = n + \operatorname{rank}\left(\mathcal{M}^{\mathcal{I}}_{i,L}\right). \tag{10}$$

Then, if the nodes run the linear iteration for $L+1$ time steps with the weight matrix $\mathcal{W}$, vehicle υ_i can calculate initial values $x_1[0], x_2[0], \ldots, x_n[0]$, even if when up to f vehicles fail to update their states correctly. □

The proof of the above theorem provides a procedure for each vehicle υ_i in the network to recover the initial conditions of all vehicles in the network, as described below.

- First, each vehicle υ_i checks all possible sets of f faulty vehicles to find set $\mathcal{I}_j$ of faulty vehicles such that $y_i[0 : L-1]$ is in the column space of $\mathcal{O}_{i,L}$ and $\mathcal{M}^{\mathcal{I}_j}_{i,L}$.
- Vehicle υ_i finds a matrix $\mathcal{N}^j_{i,L}$ whose rows form a basis for the left null space of the invertability matrix $\mathcal{M}^{\mathcal{I}_j}_{i,L}$. After finding such matrix, by left multiplying it to $\mathcal{M}^{\mathcal{I}_j}_{i,L}\boldsymbol{\zeta}[0 : L-1]$ in (6), the second term in that equation is eliminated and we have $\mathcal{N}^j_{i,L} y_i[0 : L-1] = \mathcal{N}^j_{i,L}\mathcal{O}_{i,L}$.
- The next step is to define

$$\mathcal{P}_{i,L} = \left(\mathcal{N}^j_{i,L}\mathcal{O}_{i,L}\right)^{\dagger} \mathcal{N}^j_{i,L}, \tag{11}$$

where $(.)^{\dagger}$ is the left inverse of a matrix. Then vehicle υ_i can recover the initial conditions based on its measurements after L time steps, as follows

$$\mathcal{P}_{i,L} y_i[0 : L-1] = \mathbf{x}[0]. \tag{12}$$

In Section VI we introduce graph-theoretic interpretations of Theorem 2 which are compatible with the physics of vehicle communication networks.

V. Fault Detection and Correction

In the previous section, we discussed the distributed algorithms that each vehicle can perform to obtain the required data to be used for speed fault detection and reconstruction. This section pertains to the latter steps, in which vehicle υ_i applies the data it gathered to determine if there exists a failure in its own speed measurement (or estimation).

[4]The *almost* in Theorem 1 is due to the fact that the set of parameters for which the system is not observable has Lebesgue measure zero [33].

A. Fault Detection

After finding the values for other vehicles, based on the overall procedure introduced in Section III and shown in Fig. 2, vehicle v_i performs the following error computation[5]:

$$e_i^j[kT] = \left(u_{ij}[kT] - \frac{p_{ij}[kT] - p_{ij}[(k-1)T]}{T}\right), \tag{13}$$

where $p_{ij}(kT) \triangleq p_i[kT] - p_j[kT]$ and $u_{ij}[kT] \triangleq u_i[kT] - u_j[kT]$ are relative distance and velocity of vehicles v_i and v_j, which are calculated based on the values of those quantities obtained from the distributed calculation algorithm discussed in the previous section. An (adaptive) threshold value e_{th}, obtained by road experiments and the nature of the faults that are to be detected, is used such that if $|e_i^j| > e_{th}$, it indicates the presence of a fault. For each vehicle v_i, based on the error parameter in (13) for all vehicles v_j in the network, we propose the following decision rule:

> If $|e_i^j| > e_{th}$ for only a specific vehicle $v_j \in \mathcal{V}$, then the speed u_j is faulty. If $|e_i^j| > e_{th}$ for all $v_j \in \mathcal{V}$, then the speed u_i is faulty.

B. Speed Correction

After diagnosing a fault in the speed measurement of vehicle v_i, the final step is to make a correction. This step also takes advantage of the information that v_i has obtained from the rest of the vehicles in the network. More formally, v_i calculates speed u_i from one of the following relations

$$u_i^j[kT] = u_j[kT] + \frac{p_{ij}[kT] - p_{ij}[(k-1)T]}{T}, \tag{14}$$

for each vehicle $v_j \in \mathcal{V}$, where $u_i^j[kT]$ is the speed of vehicle v_i from the perspective of vehicle v_j. Here $u_i^j[kT]$ is called the "opinion" of vehicle v_j about the velocity u_i. After calculating the opinions of all vehicles about u_i, i.e., u_i^j for all $v_j \in \mathcal{V}$, vehicle v_i can apply a majority voting rule [34] or use appropriate sensor fusion techniques to increase the reliability of the obtained values. Such post processing methods on the received signals (opinions) are necessary since these signals are prone to noise. One of these techniques is a distributed consensus algorithm to reduce the error of the resulting quantity, as discussed in [35]. However, a simpler approach (in terms of the complexity) is that vehicle v_i does a local averaging from the opinions it has computed about its speed. More specifically, vehicle v_i calculates u_i^j for all $v_j \in \mathcal{V}$, and then does the averaging as

$$\bar{u}_i = \frac{1}{n}\sum_{j=1, j\neq i}^{n} u_i^j, \tag{15}$$

where $\bar{u}_i$ is called the *average opinion* of vehicles in the network about speed u_i. At the end, v_i replaces its current velocity estimate u_i with the resulting average opinion $\bar{u}_i$.

Remark 6: In an ideal case, where all noisy opinions u_i^j are random variables with the same mean μ and variance σ^2 and are independent and identically distributed, it is well-known that the variance of the average signal $\bar{u}_i$ scales with $\frac{1}{n}$, where n is the size of the network. This shows the advantage of the resulting average opinion $\bar{u}_i$ compared to each individual opinion u_i^j. □

[5]One can use an integration of the vehicle acceleration to add another term to (13); however, integration of sensory measurements is prone to drifting effects, because of unavoidable signals bias.

Algorithm 1 `Distributed Speed Fault Diagnosis/ Correction for Vehicle` v_i

`// Inputs:` p_i, u_i, a_i and n (network size)

`Distributed Observability Matrix Computation:`
Vehicle v_i runs (4) for the i-th row of (stable) $\mathcal{W}$, and for $n - d_{min}$ time steps to calculate the observability matrix, using (9).

`Distributed Calculation:`
Vehicle v_i runs (4) for at least $L = \epsilon_i$ and at most $L = |\mathcal{S}_i| - d_i$ time steps (If $x_i[k]$ does not go to zero, then there is at least a faulty vehicle in the network).
Vehicle v_i finds matrix $\mathcal{P}_{i,L}$ via (11) and calculates initial conditions vector $\mathbf{x}[0]$ using (12).

`Fault Diagnosis:`
Vehicle v_i calculates (13). If $|e_i^j| > e_{th}$, for some e_{th}, for only a specific vehicle $v_j \in \mathcal{V}$, then speed u_j is faulty. If $|e_i^j| > e_{th}$ for all vehicles $v_j \in \mathcal{V}$, then speed u_i is faulty.,

`Fault Correction:`
Vehicle v_i uses (14) for all $v_j \in \mathcal{V} \setminus \{v_i\}$ to correct its own speed and does the local averaging (15) to reduce the signal variance.

`// Output:` Reliable and corrected speed $\bar{u}_i$.

The overall procedure of distributed calculation and fault detection and correction is summarized in Algorithm 1.

VI. Simulations and Numerical Analysis of Network Topology

In this section, some illustrative examples are demonstrated to confirm and clarify Theorems 1, 2, as well as the fault diagnosis/correction algorithms provided in the previous section. For this, we need to have more tangible (graph-theoretic) interpretations of the distributed calculation algorithm. Theorem 2 does not provide a graph-theoretic condition for state estimation. For this, the following definition is provided.

Definition 1: A vertex-cut in a graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ is a subset $\mathcal{S} \subset \mathcal{V}$ such that removing the vertices in $\mathcal{S}$ (and the associated edges) from the graph causes the remaining graph to be disconnected. More specifically, a (j,i)-cut in a graph is a subset $\mathcal{S}_{ij} \subset \mathcal{V}$ such that removing the vertices in $\mathcal{S}_{ij}$ (and the associated edges) from the graph makes the graph to have no paths from vertex v_j to vertex v_i. Let κ_{ij} denote the size of the smallest (j,i)-cut between any two vertices v_j and v_i. Then graph $\mathcal{G}$ is said to be k-connected if $\kappa_{ij} = k$. □

Theorem 3 [11]: Let $\mathcal{G}(\mathcal{V}, \mathcal{E})$ be a fixed graph and let f denote the maximum number of faulty vehicles that are to be tolerated in the network. Then, regardless of the actions of the faulty vehicles, v_i can uniquely determine all of the initial values in the network via a linear iterative strategy if $\mathcal{G}$ is at least $2f+1$ connected. □

In order to be able to apply Theorem 3, we present the following proposition.

Proposition 1: A k-nearest neighbor platoon, $\mathcal{P}(n,k)$, is a k-connected graph.

Proof: We prove using contradiction. Suppose $\mathcal{P}(n,k)$ is a $\bar{k}$-connected graph, with $\bar{k} < k$. Thus, there exists a minimum vertex cut $\mathcal{S}_{ij}$ between two vertices v_i and v_j where $|\mathcal{S}_{ij}| = \bar{k}$. Without loss of generality, label the vertices from v_i to v_j as $v_i, v_{i+1}, \ldots, v_j$. Since $\bar{k} < k$, there is a vertex $\bar{v}$ among $v_{i+1}, \ldots, v_{i+k}$ (which are directly connected to v_i) which does not belong to $\mathcal{S}_{ij}$. By replacing v_i with $\bar{v}$ in the above discussion, we will find a path from v_i to v_j which does not include vertices in $\mathcal{S}_{ij}$ and this contradicts the claim that $\mathcal{S}_{ij}$ is a vertex cut. ■

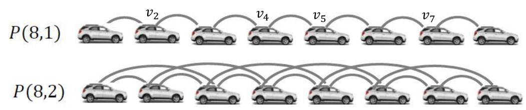

Fig. 4. Fault free 1-nearest neighbor platoon of eight vehicles (top) and 2-nearest neighbor platoon (bottom).

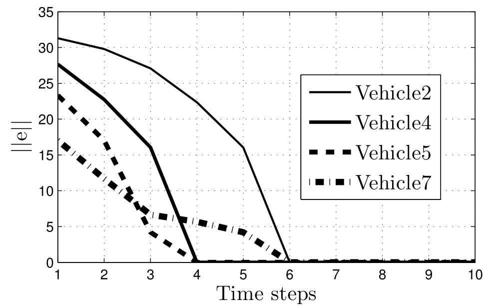

Fig. 5. Distributed calculation error for four vehicles in a 1 nearest-neighbour platoon, without fault.

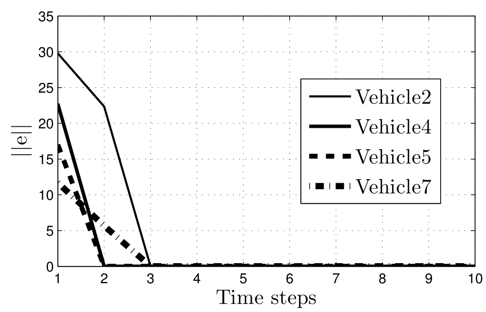

Fig. 6. Distributed calculation error for four vehicles in a 2 nearest-neighbour platoon, without fault.

Based on the above proposition, we can use $\mathcal{P}(n, k)$ as a k-connected graph in the simulations. The following figure schematically shows the vehicle network connectivity requirements for (robust) distributed calculation algorithms.

A. Fault-Free Case

For the case of fault free estimation, consider a 1 nearest-neighbour platoon (simple path graph) comprised of eight vehicles, Fig. 4 (top), in which there is no faulty vehicle in the network. The Euclidean norm of the error of the calculated quantities by vehicles 2, 4, 5, and 7 are depicted in Fig. 5. In particular, if the true initial values are denoted by vector $\mathbf{x}[0]$ and the estimation (calculation) of these initial values by each vehicle at time step k is $\hat{\mathbf{x}}[k]$, then Fig. 5 shows the Euclidean norm of the error vector $\mathbf{e}[k] = \hat{\mathbf{x}}[k] - \mathbf{x}[0]$. According to this figure, all four vehicles estimate the states in finite time. However, vehicles 4 and 5 obtain the correct values in four time steps, while vehicles 2 and 7 take six time steps. These two values for time steps show that in this network topology, the lower bound for time steps mentioned in Theorem 1 is achieved, which is equal to the longest distance of each of vehicles 2, 4, 5, and 7 to any other vehicle in the network. In Fig. 6 the connectivity of the network has increased by using a 2-nearest neighbor network of eight vehicles. In this case, vehicles 2 and 7 are two hops away from the heads of the network and vehicles 4 and 5 are only one hop away. Hence, increasing the connectivity of the network results in faster distributed calculation.

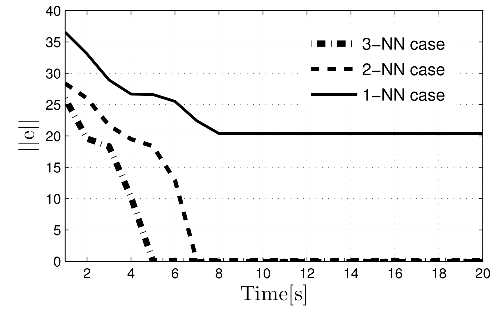

Fig. 7. Distributed calculation error for a vehicle in 1, 2 and 3 nearest-neighbour platoons, with a single fault in vehicle 3.

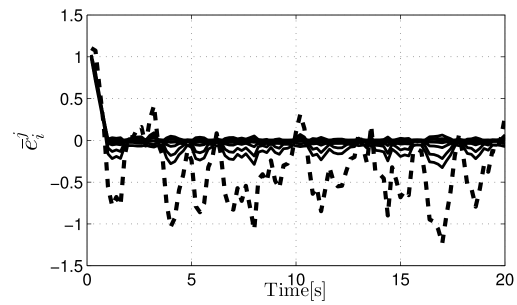

Fig. 8. Residual signals calculated by vehicle v_1.

B. Fault-Prone Case

In another scenario, we assume that there is a vehicle which fails to update its state correctly in the distributed calculation setting. Based on Theorem 3, the network should satisfy a stronger connectivity condition in order to be able to tolerate the fault. In particular, the network should be at least 3-connected such that it guarantees to tolerate one faulty vehicle. For this, we choose a larger network, which is a 3-nearest neighbor platoon of 20 vehicles. Fig. 7 shows the Euclidean norm of the error of the calculated quantities by one of the vehicles in the network v_i in the case where one of the vehicles injects a faulty input to its updating rule. As it is shown in the figure, for the cases of 2 and 3- nearest neighbor platoons, v_i manages to estimate the states, despite the existence of a faulty vehicle in the network. It should be noted that although the 2-nearest neighbor platoon does not satisfy the 3-connectivity condition, the estimation works properly. This is due to the fact that in this case, the location of the faulty vehicle and the faulty value it injects does not mislead v_i to calculate the functions. Hence, according to this figure, having a k- connected network is sufficient to overcome k faulty vehicles but not necessary. As inferred from Fig. 7, a 1-nearest neighbor platoon can no longer tolerate a faulty vehicle in the network.

C. Velocity Fault Diagnostics

After performing the distributed calculation algorithm, vehicle v_i has enough information from all other vehicles in the network to examine the correctness of its own speed. More precisely, suppose that vehicle v_i wants to determine if there exists a failure in the velocity measurement (estimation) of itself or (possibly) any other vehicle in the network. Based on the algorithm mentioned in Section V, vehicle v_i adopts an strategy to calculate the residual function $\bar{e}_i^j$ from (14) for all $v_j \in \mathcal{V} \setminus \{v_i\}$, where p_{ij} and u_j are obtained from the distributed calculation module. If $\bar{e}_i^j$ is nonzero (or above a certain threshold) for only a specific vehicle $v_j \in \mathcal{V}$, then the velocity u_j is faulty. If $\bar{e}_i^j$ is nonzero for all vehicles $v_j \in \mathcal{V}$, then the velocity u_i is faulty. Fig. 8 shows that the residual signal $\bar{e}_i^j$, computed by vehicle v_1, i.e., $i = 1$, in $\mathcal{P}(8, 2)$, for vehicle v_3 (dashed line) is significantly larger than those of the rest of the vehicles in the network. Hence, the velocity of vehicle v_3, which is u_3, is faulty. In this case, v_3 should calculate the opinions of the other seven vehicles in the network about its own speed, i.e., u_3^j for all $v_j \in \mathcal{V}$. Vehicle v_3 has the required materials for doing this calculation, which are all p_i and u_i for all other vehicles, as it has gathered from the distributed calculation algorithm. After calculating all opinions u_3^j, v_3 does an averaging between the opinions and replaces its current velocity u_3 with the resulting velocity $\bar{u}_3$.

VII. Summary and Conclusions

This paper introduced a cooperative vehicle longitudinal velocity (speed) fault detection and correction algorithm. To perform fault detection and correction algorithm, each vehicle needs to gather some information from other vehicles in the network, including speed and position. Hence, a distributed function calculation strategy is used for each vehicle to gather this information from the network in a distributed manner. Then every vehicle operates a specific fault diagnosis algorithm to find out if there exists a failure in its own velocity estimation (or measurement) and correct it. Several simulation results presented to validate the theoretical results. The proposed algorithm, in a nutshell, makes a bridge between the estimated velocity of each vehicle and information coming from other vehicles in a network to come up with a milestone for assessing the correctness and reliability of the estimated velocity by each vehicle. From the implementation point of view, the algorithm is easily applicable to a network of connected vehicles and is compatible with the current standards for the inter-vehicular communications (discussed in Section II-B). In addition, the distributed calculation algorithm used for gathering information from distant vehicles in the network is robust to common communication faults and drops (which happen frequently in tunnels, called the *tunnel mode* [36]). Hence, applying the proposed fault detection and identification algorithm is quite feasible in real-world vehicular networks.

References

[1] N. Lu, N. Cheng, N. Zhang, X. Shen, and J. W. Mark, "Connected vehicles: Solutions and challenges," *IEEE Internet Things J.*, vol. 1, no. 4, pp. 289–299, Aug. 2014.

[2] N. Liu, "Internet of vehicles: Your next connection," *Huawei WinWin*, vol. 11, pp. 23–28, Dec. 2011.

[3] J. A. Stankovic, "Research directions for the Internet of Things," *IEEE Internet Things J.*, vol. 1, no. 1, pp. 3–9, Feb. 2014.

[4] N.-E. El Faouzi, H. Leung, and A. Kurian, "Data fusion in intelligent transportation systems: Progress and challenges—A survey," *Inf. Fusion*, vol. 12, no. 1, pp. 4–10, 2011.

[5] M. Tubaishat, P. Zhuang, Q. Qi, and Y. Shang, "Wireless sensor networks in intelligent transportation systems," *Wireless Commun. Mobile Comput.*, vol. 9, no. 3, pp. 287–302, 2008.

[6] M. Patra, R. Thakur, and C. S. R. Murthy, "Improving delay and energy efficiency of vehicular networks using mobile femto access points," *IEEE Trans. Veh. Technol.*, vol. 66, no. 2, pp. 1496–1505, Feb. 2017.

[7] S. Antonov, A. Fehn, and A. Kugi, "Unscented Kalman filter for vehicle state estimation," *Vehicle Syst. Dyn.*, vol. 49, no. 9, pp. 1497–1520, 2011.

[8] M. Pirani *et al.*, "Resilient corner-based vehicle velocity estimation," *IEEE Trans. Control Syst. Technol.*, vol. 26, no. 2, pp. 452–462, Mar. 2018.

[9] T. A. Wenzel, K. J. Burnham, M. V. Blundell, and R. A. Williams, "Dual extended Kalman filter for vehicle state and parameter estimation," *Vehicle Syst. Dyn.*, vol. 44, no. 2, pp. 153–171, 2006.

[10] E. Hashemi, M. Pirani, A. Khajepour, A. Kasaiezadeh, S.-K. Chen, and B. Litkouhi, "Corner-based estimation of tire forces and vehicle velocities robust to road conditions," *Control Eng. Pract.*, vol. 61, pp. 28–40, Apr. 2017.

[11] S. Sundaram and C. N. Hadjicostis, "Distributed function calculation via linear iterative strategies in the presence of malicious agents," *IEEE Trans. Autom. Control*, vol. 56, no. 7, pp. 1495–1508, Jul. 2011.

[12] R. Olfati-Saber and J. S. Shamma, "Consensus filters for sensor networks and distributed sensor fusion," in *Proc. 44th IEEE Conf. Decision Control*, Dec. 2005, pp. 6698–6703.

[13] R. Olfati-Saber, "Kalman-consensus filter: Optimality, stability, and performance," in *Proc. IEEE Conf. Decision Control*, Dec. 2009, pp. 7036–7042.

[14] I. D. Schizas, A. Ribeiro, and G. B. Giannakis, "Consensus in *Ad Hoc* WSNs with noisy links—Part I: Distributed estimation of deterministic signals," *IEEE Trans. Signal Process.*, vol. 58, no. 1, pp. 350–364, Jan. 2008.

[15] M. Kamgarpour and C. Tomlin, "Convergence properties of a decentralized Kalman filter," in *Proc. IEEE Conf. Decision Control*, 2008, pp. 3205–3210.

[16] H. Fawzi, P. Tabuada, and S. Diggavi, "Secure estimation and control for cyber-physical systems under adversarial attacks," *IEEE Trans. Autom. Control*, vol. 59, no. 6, pp. 1454–1467, Jun. 2014.

[17] S. Sundaram and C. N. Hadjicostis, "Distributed function calculation and consensus using linear iterative strategies," *IEEE J. Sel. Areas Commun.*, vol. 26, no. 4, pp. 650–660, May 2008.

[18] V. Borkar and P. P. Varaiya, "Asymptotic agreement in distributed estimation," *IEEE Trans. Autom. Control*, vol. AC-27, no. 3, pp. 650–655, Jun. 1982.

[19] F. S. Cattivelli and A. H. Sayed, "Diffusion LMS strategies for distributed estimation," *IEEE Trans. Signal Process.*, vol. 58, no. 3, pp. 1035–1048, Mar. 2010.

[20] A. Mitra and S. Sundaram, "An approach for distributed state estimation of LTI systems," in *Proc. 54th Annu. Allerton Conf. Commun., Control, Comput. (Allerton)*, Sep. 2016, pp. 1088–1093.

[21] A. Speranzon, C. Fischione, and K. H. Johansson, "Distributed and collaborative estimation over wireless sensor networks," in *Proc. 45th IEEE Conf. Decision Control*, Dec. 2006, pp. 1025–1030.

[22] M. Pirani, E. Hashemi, J. W. Simpson-Porco, B. Fidan, and A. Khajepour, "Graph theoretic approach to the robustness of k-nearest neighbor vehicle platoons," *IEEE Trans. Intell. Transp. Syst.*, vol. 18, no. 11, pp. 3218–3224, Nov. 2017.

[23] H. J. Miller and S.-L. Shaw, *Geographic Information Systems for Transportation*. Oxford, U.K.: Oxford Univ. Press, 2001.

[24] *IEEE Standard for Wireless Access in Vehicular Environments (WAVE)—Identifier Allocations*, IEEE Standard 1609.12-2016, U.D. Transportation, 2013.

[25] C. Hedges and F. Perry, "Overview and use of SAE J2735 message sets for commercial vehicles," SAE Tech. Paper 2008-01-2650, 2008.

[26] J. J. Anaya, E. Talavera, F. Jiménez, N. Gómez, and J. E. Naranjo, "A novel geo-broadcast algorithm for V2V communications over WSN," *Electronics*, vol. 3, no. 3, pp. 521–537, 2014.

[27] M. Pirani and S. Sundaram, "On the smallest eigenvalue of grounded Laplacian matrices," *IEEE Trans. Autom. Control*, vol. 61, no. 2, pp. 509–514, Feb. 2016.

[28] M. Pirani, E. M. Shahrivar, B. Fidan, and S. Sundaram, "Robustness of leader-follower networked dynamical systems," *IEEE Trans. Control Netw. Syst.*, to be published.

[29] C. N. Hadjicostis and R. Touri, "Feedback control utilizing packet dropping network links," in *Proc. 41st IEEE Conf. Decision Control*, Dec. 2002, pp. 1205–1210.

[30] P. Seiler and R. Sengupta, "Analysis of communication losses in vehicle control problems," in *Proc. Amer. Control Conf.*, Jun. 2001, pp. 1491–1496.

[31] J. P. Hespanha, P. Naghshtabrizi, and Y. Xu, "A survey of recent results in networked control systems," *Proc. IEEE*, vol. 95, no. 1, pp. 138–162, Jan. 2007.

[32] C.-T. Chen, *Linear System Theory and Design*. New York, NY, USA: Holt, 1984.

[33] K. Reinschke, *Multivariable Control a Graph-theoretic Approach*. Berlin, Germany: Springer, 1987.

[34] D. Peleg, "Local majority voting, small coalitions and controlling monopolies in graphs: A review," in *Proc. 3rd Colloq. Struct. Inf. Commun. Complex.*, 1997, pp. 152–169.

[35] L. Xiao, S. Boyd, and S. Lall, "A scheme for robust distributed sensor fusion based on average consensus," in *Proc. 4th Int. Symp. Inf. Process. Sensor Netw.*, Apr. 2005, pp. 63–70.

[36] P. Srisuresh, "Security model with tunnel-mode IPsec for NAT domains," document RFC 2709, Lucent Technologies, Oct. 1999.